What Is Prediction?

- Prediction is different from classification
  - Classification refers to predict categorical class label
  - Prediction models continuous-valued functions
    - To predict the salary of college graduates with 10 years of work experience.
    - The potential sales of a new product given its price.
- Major method for prediction: regression
  - A statistical methodology
  - model the relationship between one or more *independent* or **predictor** variables and a *dependent* or **response** variable.
  - Given a tuple described by predictor variables, we want to predict the associated value of the response variable.
- Regression analysis
  - Linear and multiple regression
  - Non-linear regression

Linear Regression

- Linear regression: involves a response variable y and a single predictor variable x
- It is the simplest form of regression, and models *y* as a linear function of *x*.

$$y = w_0 + w_1 x$$

where w_0 (y-intercept) and w_1 (slope) are regression coefficients

- These coefficients can be solved for by the method of least squares, which estimates the best-fitting straight line as the one that minimizes the error between the actual data and the estimate of the line.
- Let *D* be a training set consisting of values of predictor variable, *x*, for some population and their associated values for response variable, *y*.
- The training set contains |*D*| data points of the form(x_1, y_1), (x_2, y_2), …. , $(x_{|D|}, y_{|D|})$

Linear Regression

- The regression coefficients can be estimated using least square method with the following equations:

$$w_1 = \frac{\sum_{i=1}^{|D|}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{|D|}(x_i - \bar{x})^2}$$

$$w_0 = \bar{y} - w_1\bar{x}$$

  - where $\bar{x}$ is the mean value of $x_1, x_2, \ldots, x_{|D|}$, and $\bar{y}$ is the mean value of $y_1, y_2, \ldots, y_{|D|}$.
  - Table 6.7 shows a set of paired data where x is the number of years of work experience of a college graduate and y is the corresponding salary of the graduate.

Linear Regression

- We model the relationship that salary may be related to the number of years of work experience with the equation

  $y = w_0 + w_1 x$.

- we compute $\bar{x} = 9.1$ and $\bar{y} = 55.4$.
- Substituting these values into Equations , we get

Table 6.7 Salary data.

x years experience	*y* salary (in $1000s)
3	30
8	57
9	64
13	72
3	36
6	43
11	59
21	90
1	20
16	83

Linear Regression

$$w_1 = \frac{(3-9.1)(30-55.4)+(8-9.1)(57-55.4)+\cdots+(16-9.1)(83-55.4)}{(3-9.1)^2+(8-9.1)^2+\cdots+(16-9.1)^2} = 3.5$$

$$w_0 = 55.4-(3.5)(9.1) = 23.6$$

- The equation of the least squares line is estimated by $y = 23.6 + 3.5x$.
- Using this equation, we can predict that the salary of a college graduate with 10 years of experience is $58,600.

Linear Regression

- Multiple linear regression:
  - It is an extension of straight-line regression so as to involve more than one predictor variable.
  - It allows response variable y to be modeled as a linear function of n predictor variables or attributes, A_1, A_2, … , A_n, describing a tuple, $\boldsymbol{X}$.
  - Our training data set, D, contains data of the form $(\boldsymbol{X_1}, y_1)$, $(\boldsymbol{X_2}, y_2)$, … , $(\boldsymbol{X}_{|\boldsymbol{D}|}, y_{|D|})$, where the $\boldsymbol{X_i}$ are the n-dimensional training tuples with associated class labels, y_i.
  - An example of a multiple linear regression model based on two predictor attributes or variables, A_1 and A_2, where x_1 and x_2 are the values of attributes A_1 and A_2, respectively, in $\boldsymbol{X}$.
  - The method of least squares shown above can be extended to solve for w_0, w_1, and w_2.

Nonlinear Regression

- Some nonlinear models can be modeled by a polynomial function
- A polynomial regression model can be transformed into linear regression model. For example,

$$y = w_0 + w_1 x + w_2 x^2 + w_3 x^3$$

convertible to linear with new variables: $x_2 = x^2$, $x_3 = x^3$

$$y = w_0 + w_1 x + w_2 x_2 + w_3 x_3$$

- Other functions, such as power function, can also be transformed to linear model
- Some models are intractable nonlinear (e.g., sum of exponential terms)
  - possible to obtain least square estimates through extensive calculation on more complex formulae

Chapter 6. Classification and Prediction

- What is classification? What is prediction?
- Issues regarding classification and prediction
- Classification by decision tree induction
- Bayesian classification
- Rule-based classification
- Classification by back propagation
- Support Vector Machines (SVM)
- Associative classification
- Lazy learners (or learning from your neighbors)
- Other classification methods
- Prediction
- Accuracy and error measures
- Ensemble methods
- Model selection
- Summary

Classifier Accuracy Measures

- The accuracy of a classifier on a given test set is the percentage of test set tuples that are correctly classified by the classifier.
- It reflects how well the classifier recognizes tuples of the various classes.
- The *confusion matrix* is a useful tool for analyzing how well your classifier can recognize tuples of different classes.

classes	buy_computer = yes	buy_computer = no	total	recognition(%)
buy_computer = yes	6954	46	7000	99.34
buy_computer = no	412	2588	3000	86.27
total	7366	2634	10000	95.52

Classifier Accuracy Measures

- Given *m* classes, a confusion matrix is a table of at least size *m* by *m*.
- An entry, $CM_{i,j}$ in the first *m* rows and *m* columns indicates the number of tuples of class *i* that were labeled by the classifier as class *j*.
- For a classifier to have good accuracy, ideally most of the tuples would be represented along the diagonal of the confusion matrix, from entry $CM_{1,1}$ to entry $CM_{m,m}$ with the rest of the entries being close to zero.
- The table may have additional rows or columns to provide totals or recognition rates per class.

Classifier Accuracy Measures

		Predicted class	
		C_1	C_2
Actual class	C_1	true positives	false negatives
	C_2	false positives	true negatives

- TP (Eg "cancer samples" that were correctly classified as such)
- TN ("not_cancer" samples that were correctly classified as such)
- FP ("not_cancer" samples that were incorrectly labeled as "cancer")
- FN ("cancer" samples that were incorrectly labeled as "not_cancer")
- P is the number of positive samples
- N is the number of negative samples

Classifier Accuracy Measures

- The sensitivity and specificity measures can be used to access how well the classifier can recognize *"cancer"* tuples and how well it can recognize *"not cancer"* tuples, respectively.
- Sensitivity is also referred to as the *true positive rate*
  - sensitivity = TP/P
- Specificity is the *true negative rate*
  - specificity = TN/N
- we may use precision to access the percentage of tuples labeled as *"cancer"* that actually are *"cancer"* tuples.
  - precision = TP/(TP + FP)
- Accuracy is a function of sensitivity and specificity:
  - accuracy = sensitivity * P/(P+N) + specificity * N/(P+N)

Classifier Accuracy Measures

Classes	*yes*	*no*	*Total*	*Recognition (%)*
yes	90	210	300	30.00
no	140	9560	9700	98.56
Total	230	9770	10,000	96.40

Confusion matrix for the classes *cancer* = *yes* and *cancer* = *no*.

- The sensitivity of the classifier is 90 / 300 = 30.00%.
- The specificity is 9560 / 9700 = 98.56%.
- The classifier's overall accuracy is 9650 / 10,000 = 96.50%.
- Although the classifier has a high accuracy, it's ability to correctly label the positive class is poor given its low sensitivity.
- It has high specificity, meaning that it can accurately recognize negative tuples.

Predictor Error Measures

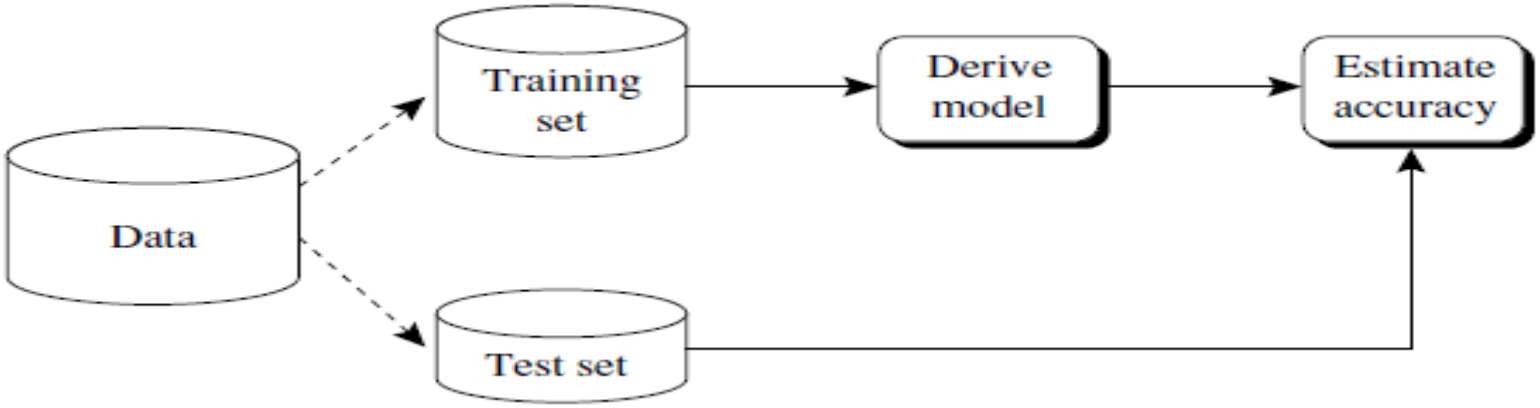
- Measure predictor accuracy: measure how far off the predicted value is from the actual known value
- **Loss function**: measures the error betw. y_i and the predicted value y_i'
  - Absolute error: $| y_i - y_i' |$
  - Squared error: $(y_i - y_i')^2$
- Test error (generalization error): the average loss over the test set
  - Mean absolute error: $\frac{\sum_{i=1}^{d} | y_i - y_i' |}{d}$ Mean squared error: $\frac{\sum_{i=1}^{d} (y_i - y_i')^2}{d}$
  - Relative absolute error: $\frac{\sum_{i=1}^{d} | y_i - y_i' |}{\sum_{i=1}^{d} | y_i - \bar{y} |}$ Relative squared error: $\frac{\sum_{i=1}^{d} (y_i - y_i')^2}{\sum_{i=1}^{d} (y_i - \bar{y})^2}$

  The mean squared-error exaggerates the presence of outliers

  Popularly use (square) root mean-square error, similarly, root relative squared error

Holdout Method and Random Sampling

- Holdout method
  - Given data is randomly partitioned into two independent sets
    - Training set (e.g., 2/3) for model construction
    - Test set (e.g., 1/3) for accuracy estimation



  - Random sampling: a variation of holdout
    - Repeat holdout k times, accuracy = avg. of the accuracies obtained

Cross-Validation

- Cross-validation (*k*-fold, where k = 10 is most popular)
  - Randomly partition the data into *k mutually exclusive* subsets, each approximately equal size
  - At *i*-th iteration, use D_i as test set and others as training set
  - The accuracy estimate =

$$\frac{\text{Overall number of correct classifications from the k iterations}}{\text{Total number of samples in the initial data}}$$

  - Leave-one-out: k folds where k = # of tuples, for small sized data
  - Stratified cross-validation: folds are stratified so that class dist. in each fold is approx. the same as that in the initial data

Bootstrap

- Bootstrap
  - Works well with small data sets
  - Samples the given training tuples uniformly *with replacement*
    - i.e., each time a tuple is selected, it is equally likely to be selected again and re-added to the training set
- Several boostrap methods, and a common one is **.632 boostrap**
  - Suppose we are given a data set of d tuples. The data set is sampled d times, with replacement, resulting in a training set of d samples.
  - The data tuples that did not make it into the training set end up forming the test set.
  - About 63.2% of the original data will end up in the bootstrap, and the remaining 36.8% will form the test set (since $(1 - 1/d)^d \approx e^{-1} = 0.368$)

Bootstrap

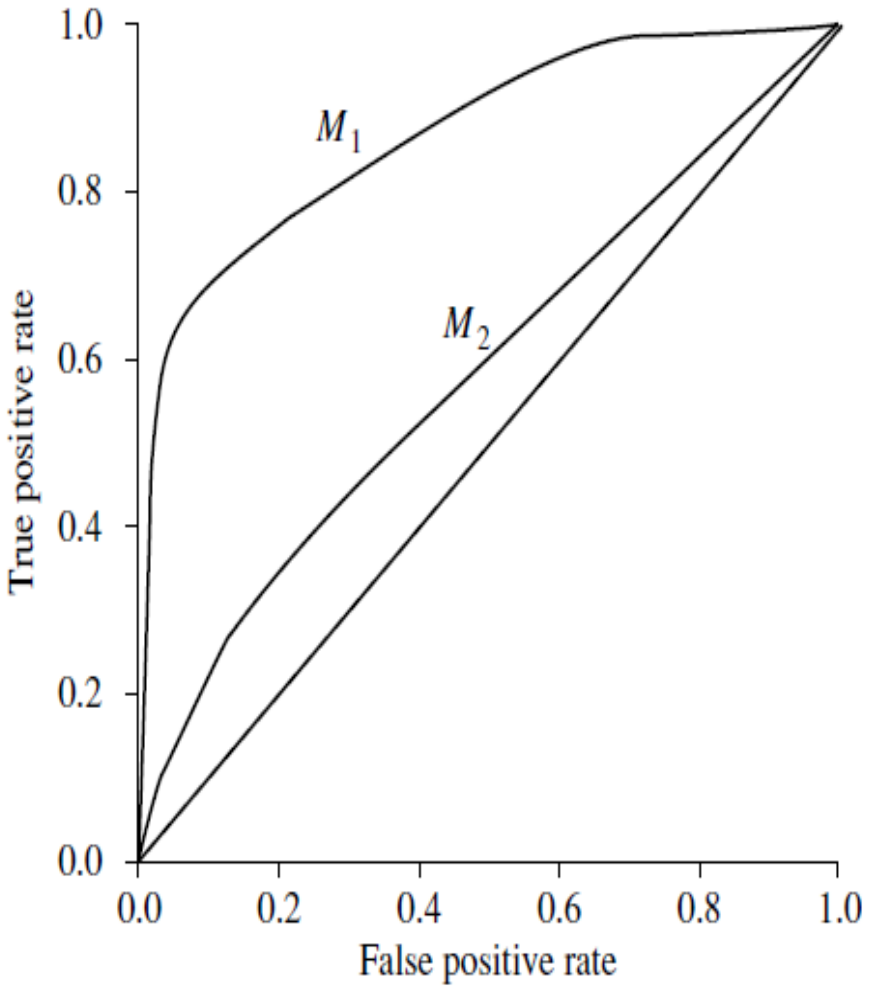
- Each tuple has a probability of 1/ *d* of being selected, so the probability of not being chosen is 1-1/*d*.
- We have to select *d* times, so the probability that a tuple will not be chosen during this whole time is $(1-1/d)^d$.
- If *d* is large, the probability approaches $e^{-1} = 0.368$.[7] Thus, 36.8% of tuples will not be selected for training and thereby end up in the test set, and the remaining 63.2% will form the training set.
- Repeat the sampling procedure k times, overall accuracy of the model:

Model Selection: ROC Curves

- ROC (Receiver Operating Characteristics) curves: for visual comparison of classification models
- Originated from signal detection theory
- Shows the trade-off between the true positive rate and the false positive rate
- The area under the ROC curve is a measure of the accuracy of the model
- Rank the test tuples in decreasing order: the one that is most likely to belong to the positive class appears at the top of the list
- The closer to the diagonal line (i.e., the closer the area is to 0.5), the less accurate is the model

Model Selection: ROC Curves

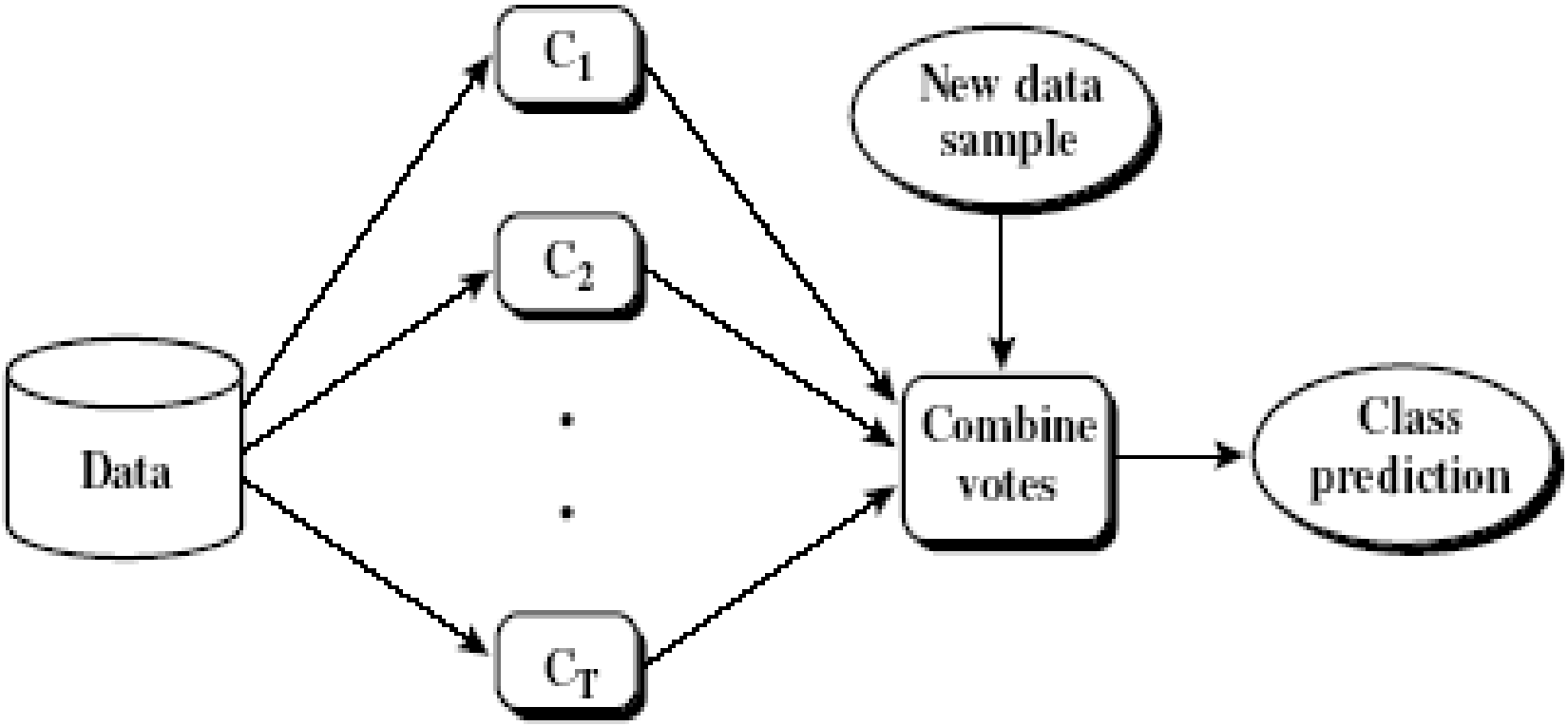
- Vertical axis represents the true positive rate
- Horizontal axis rep. the false positive rate
- The plot also shows a diagonal line
- A model with perfect accuracy will have an area of 1.0



Chapter 6. Classification and Prediction

- What is classification? What is prediction?
- Issues regarding classification and prediction
- Classification by decision tree induction
- Bayesian classification
- Rule-based classification
- Classification by back propagation
- Support Vector Machines (SVM)
- Associative classification
- Lazy learners (or learning from your neighbors)
- Other classification methods
- Prediction
- Accuracy and error measures
- Ensemble methods
- Model selection
- Summary

Ensemble Methods: Increasing the Accuracy



Ensemble Methods: Increasing the Accuracy

- Ensemble methods
  - Use a combination of models to increase accuracy
  - Combine a series of k learned models, M_1, M_2, …, M_k, with the aim of creating an improved model M*
- Popular ensemble methods
  - Bagging: averaging the prediction over a collection of classifiers
  - Boosting: weighted vote with a collection of classifiers
  - Random Forest : Each classifier in the ensemble is a *decision tree* classifier

Bagging: Boostrap Aggregation

- Analogy: Diagnosis based on multiple doctors' majority vote
- Training
  - Given a set D of *d* tuples, at each iteration *i*, a training set D_i of *d* tuples is sampled with replacement from D (i.e., boostrap)
  - A classifier model M_i is learned for each training set D_i
- Classification: classify an unknown sample **X**
  - Each classifier M_i returns its class prediction
  - The bagged classifier M* counts the votes and assigns the class with the most votes to **X**
- Prediction: can be applied to the prediction of continuous values by taking the average value of each prediction for a given test tuple
- Accuracy
  - Often significant better than a single classifier derived from D
  - For noise data: not considerably worse, more robust
  - Proved improved accuracy in prediction

Algorithm: Bagging

The bagging algorithm—create an ensemble of classification models for a learning scheme where each model gives an equally weighted prediction.

Input:

- *D*, a set of *d* training tuples;
- *k*, the number of models in the ensemble;
- A classification learning scheme (decision tree algorithm, na¨ıve Bayesian, etc.).

Output: The ensemble—a composite model, *M*.

Method:

(1) **for** *i* = 1 to *k* **do** // create *k* models:

(2) create bootstrap sample, D_i , by sampling *D* with replacement;

(3) use D_i and the learning scheme to derive a model, M_i;

(4) **endfor**

To use the ensemble to classify a tuple, *X*:

let each of the *k* models classify ***X*** and return the majority vote;

Boosting

- Analogy: Consult several doctors, based on a combination of weighted diagnoses—weight assigned based on the previous diagnosis accuracy
- How boosting works?
  - Weights are assigned to each training tuple
  - A series of k classifiers is iteratively learned
  - After a classifier M_i is learned, the weights are updated to allow the subsequent classifier, M_{i+1}, to pay more attention to the training tuples that were misclassified by M_i
  - The final M* combines the votes of each individual classifier, where the weight of each classifier's vote is a function of its accuracy
- The boosting algorithm can be extended for the prediction of continuous values
- Comparing with bagging: boosting tends to achieve greater accuracy, but it also risks overfitting the model to misclassified data

Adaboost (Adaptive Boosting)

- Given a set of *d* class-labeled tuples, ($\mathbf{X_1}$, y_1), …, ($\mathbf{X_d}$, y_d)
- Initially, all the weights of tuples are set the same (1/d)
- Generate k classifiers in k rounds. At round i,
  - Tuples from D are sampled (with replacement) to form a training set D_i of the same size
  - Each tuple's chance of being selected is based on its weight
  - A classification model M_i is derived from D_i
  - Its error rate is calculated using D_i as a test set
  - If a tuple is misclssified, its weight is increased, o.w. it is decreased
- Error rate: err($\mathbf{X_j}$) is the misclassification error of tuple $\mathbf{X_j}$. Classifier M_i error rate is the sum of the weights of the misclassified tuples:

$$error(M_i) = \sum_{j}^{d} w_j \times err(\mathbf{X_j})$$

Adaboost (Adaptive Boosting)

- If the tuple was misclassified, then err(Xj) is 1; otherwise, it is 0.
- If the performance of classifier M_i is so poor that its error exceeds 0.5, then we abandon it. Instead, we try again by generating a new D_i training set, from which we derive a new M_i
- The error rate of M_i affects how the weights of the training tuples are updated.
- If a tuple in round i was correctly classified, its weight is multiplied by error(M_i)= (1- error(M_i)).
- Once the weights of all the correctly classified tuples are updated, the weights for all tuples are normalized
- To normalize a weight, we multiply it by the sum of the old weights, divided by the sum of the new weights.
- The weights of misclassified tuples are increased and the weights of correctly classified tuples are decreased.

Adaboost (Adaptive Boosting)

- Boosting assigns a weight to each classifier's vote, based on how well the classifier performed.
- The lower a classifier's error rate, the more accurate it is.
- The weight of classifier *Mi*'s vote is

$$\log \frac{1-error(M_i)}{error(M_i)}$$

- For each class, *c*, we sum the weights of each classifier that assigned class *c* to ***X***.
- The class with the highest sum is the "winner" and is returned as the class prediction for tuple ***X***.

Algorithm: AdaBoost. A boosting algorithm—create an ensemble of classifiers. Each one gives a weighted vote.

Input:

- D, a set of d class-labeled training tuples;
- k, the number of rounds (one classifier is generated per round);
- a classification learning scheme.

Output: A composite model.

Method:

(1) initialize the weight of each tuple in D to $1/d$;
(2) **for** $i = 1$ to k **do** // for each round:
(3) sample D with replacement according to the tuple weights to obtain D_i;
(4) use training set D_i to derive a model, M_i;
(5) compute $error(M_i)$, the error rate of M_i (Eq. 8.34)
(6) **if** $error(M_i) > 0.5$ **then**
(7) go back to step 3 and try again;
(8) **endif**
(9) **for** each tuple in D_i that was correctly classified **do**
(10) multiply the weight of the tuple by $error(M_i)/(1 - error(M_i))$; // update weights
(11) normalize the weight of each tuple;
(12) **endfor**

To use the ensemble to classify tuple, X:

(1) initialize weight of each class to 0;

(2) for $i = 1$ to k do // for each classifier:

(3) $w_i = log \frac{1-error(M_i)}{error(M_i)}$; // weight of the classifier's vote

(4) $c = M_i(X)$; // get class prediction for X from M_i

(5) add w_i to weight for class c

(6) endfor

(7) return the class with the largest weight;

Random Forest (Breiman 2001)

- Random Forest:
  - Each classifier in the ensemble is a *decision tree* classifier and is generated using a random selection of attributes at each node to determine the split
  - During classification, each tree votes and the most popular class is returned
- Two Methods to construct Random Forest:
  - Forest-RI (*random input selection*): Randomly select, at each node, F attributes as candidates for the split at the node. The CART methodology is used to grow the trees to maximum size
  - Forest-RC (*random linear combinations*): Creates new attributes (or features) that are a linear combination of the existing attributes (reduces the correlation between individual classifiers)
- Comparable in accuracy to Adaboost, but more robust to errors and outliers
- Insensitive to the number of attributes selected for consideration at each split, and faster than bagging or boosting